Statistics Basic

LAB 10 - 로지스틱 회귀

6. [PBT] Apple Quality

ITWILL 이광호 강사

4. 가정 확인

모든 실습이 하나의 노트북 파일에서 이어서 진행됩니다.

실습량이 매우매우매우매우 많습니다.

ITWILL 이광호 강사

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



단계 5: 기준선 모형

□ 전처리를 하나도 적용하지 않은 상태에서 먼저 적합한다.

STEP 1
종속변수 라벨링
문자열 → 0/1

▶

STEP 2
기준선 모형 적합

▶

STEP 3
변수 보고 출력

라벨링 처리시 주의점 — 어느 범주가 1이 되는가

라벨링 함수는 **알파벳 순서로 0부터** 부여한다.
어느 범주가 1이 되는지 확인해야 이후 해석의 기준이 정해진다 — 확인 없이 넘어가면 단계 8의 해석 방향이 뒤집힌다.

순서를 어기면

다음 단계의 가정 검정이 이 모형을 대상으로 한다.
전처리를 먼저 적용해 버리면 무엇을 진단한 것인지 특정할 수 없게 된다.

실행 규칙

여기서 **유의하지 않게 나온 변수도 아직 지우지 않는다.**
이 단계의 출력은 제거 판정용이 아니라, **단계 6 가정 검정의 입력**으로 쓰인다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



이전 작업 과정 로드

실습

□ 새로운 파일에서 코드 작성시 기존 파일과 흐름이 이어지도록 구성

```python
%%capture cap
%run "./[LAB-10] 로지스틱 회귀 | 06-[PBT] Apple Quality(1)(실습코드).ipynb"
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#08. 로지스틱 회귀 기준선 모형

실습

1. 종속변수 라벨링

◎ 로지스틱 회귀의 종속변수는 **0/1의 정수**여야 하므로 문자열 **good/bad**를 숫자로 바꾼다.

○ **my_prep.labeling()**은 **알파벳 순서**로 0부터 매긴다. → **bad = 0, good = 1**

◎ 즉 이 모형이 예측하는 것은 **P(Quality = good)** (사과의 품질이 good일 확률P)이고,
뒤에 나오는 오즈비는 모두 **"사과가 `good` 일 오즈"** 에 대한 값이다.

```python
base = my_prep.labeling(df, columns=[target])
base.head()
```

Python

```
Quality (2종): bad=0, good=1
```

	Size	Weight	Sweetness	Crunchiness	Juiciness	Ripeness	Acidity	Quality
0	-3.970	-2.512	5.346	-1.012	1.845	0.330	-0.492	1
1	-1.195	-2.839	3.664	1.588	0.853	0.868	-0.723	1
2	-0.292	-1.351	-1.738	-0.343	2.839	-0.038	2.622	0
3	-0.657	-2.272	1.325	-0.098	3.638	-3.414	0.791	1
4	1.364	-1.297	-0.385	-0.553	3.031	-1.304	0.502	1

인사이트는 다음 페이지에 제시합니다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#08. 로지스틱 회귀 기준선 모형

□ 인사이트 — EDA에서 탈락했던 변수가 되살아났다

변수	T검정 (단독 비교)	기준선 모형 (다른 변수 통제)	판정
Size	$p < 0.001$	$z = 22.99, p < 0.001$	그대로 유의
Sweetness	$p < 0.001$	$z = 20.94, p < 0.001$	그대로 유의
Juiciness	$p < 0.001$	$z = 18.24, p < 0.001$	그대로 유의
Ripeness	$p < 0.001$	$z = -5.13, p < 0.001$	그대로 유의
Acidity	**$p = 0.797$ (무관)**	**$z = -13.70, p < 0.001$**	**되살아남**
Weight	**$p = 0.928$ (무관)**	**$z = 9.40, p < 0.001$**	**되살아남**
Crunchiness	**$p = 0.985$ (무관)**	**$p = 0.253$**	**유일한 탈락 후보**

- T검정에서 "차이가 전혀 없다"고 나왔던 **Acidity와 Weight가 $p < 0.001$로 강하게 유의**해졌다 — 이것이 **억제효과 (suppression)**다. 다른 변수를 통제하지 않은 단독 비교에서는 효과가 가려져 있었던 것이다.
- Acidity의 오즈비는 **0.745**로, 산도가 1점 높아지면 good일 오즈가 **약 25.5% 감소**한다.
- **EDA에서 유의하지 않다는 이유만으로 변수를 버리면 안 된다.**
- 유의하지 않은 변수는 Crunchiness 하나뿐이지만, **여기서도 아직 지우지 않는다.**

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



단계 6: 가정 검정 루프

□ 로지스틱 회귀에서 점검하는 가정은 로짓 선형성과 독립성 두 가지

◎ 선형회귀의 정규성·등분산성 항목은 대상이 아니다.

◎ **이 파이프라인의 핵심 — 가정 검정은 한 번으로 끝나지 않고 루프를 돈다.**

진단
로짓 선형성 검정을 전체 연속형 변수에 실행

▶

처방
위배가 가장 심한 변수 하나만 제곱항과 로그 변환을 모두 적용

▶

검증
제곱항은 우도비 검정으로, 로그 변환은 AIC로 판정

▶

재진단
손대지 않은 변수만 다시 검정

↺ **한 라운드에 한 변수씩** — 위배 변수가 사라질 때까지 진단·처방·검증·재진단을 반복한다

검증 단계를 두는 이유

모수를 추가하면 **적합도는 언제나 좋아진다** — 개선이 **우연 이상인지 확인한 뒤에야** 처방을 확정한다.

재진단을 손대지 않은 변수만 하는 이유

한 변수를 고치면 다른 변수의 진단 결과가 바뀐다. 그래서 재진단이 필요하다.
이때 **처방을 마친 변수는 진단 대상에서 완전히 제외**한다 — 파생항을 다시 검정하면 판정이 성립하지 않기 때문이다.

둘 다 검증을 통과하면 AIC가 낮은 쪽을 채택한다. 제곱항은 단조 곡선도 근사하므로, “제곱항이 유의하지 않을 때만 로그”로 두면 **로그가 맞는 형태여도 제곱항이 먼저 뽑힌다**. 제곱항이 기각되면 자연히 로그가 채택된다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



단계 6: 루프 코딩 주의점

□ 단계 6의 코딩 주의점 세 가지

주의 1 원본을 그대로 제곱하지 않는다

중심화(평균 빼기) 후 제곱한다.
원본을 그대로 제곱하면 1차항과의 상관이 극단적으로 높아져, 두 계수의 추정이 불안정해진다.

주의 2 처방 기준값을 반드시 보관한다

중심화에 쓴 평균과 로그 변환의 **평행이동량**은 **예측 단계에서 그대로 재사용**해야 한다.
신규 데이터로 다시 계산하면 기준이 어긋나 단계 8의 예측 절차가 성립하지 않는다.

주의 3 로그 변환은 x > 0일 때만 시도한다

음수가 있으면 평행이동해야 하는데, **이동량이 자의적이어서 계수 해석이 무너진다.**
로그를 쓸 수 없는 데이터에서는 **제곱항이 사실상 유일한 선택지**가 된다.

처방 결과가 다음 단계의 입력이 된다

단계 6에서 만든 **중심화·제곱항**과 **보관한 평균값**은 단계 7 파이프라인 함수의 **로짓 선형성 처방 옵션**으로 그대로 들어간다.

즉 단계 6은 진단만 하는 자리가 아니라, **단계 7이 쓸 파생변수를 만들어 넘기는 자리**다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-1)

실습

□ 로짓 선형성 검정 함수 개선 (1/2)

◎ 다른 함수에서 활용할 수 있도록 파라미터 추가

```
1269 def test_linear(fit=None, data=None, yname=None, xnames=None, targets=None, alpha=0.05):
1270     """Box-Tidwell 검정으로 연속형 독립변수의 로짓 선형성을 점검한다.
1271
1272     독립변수 x에 `x × ln(x)` 항을 하나씩 추가해 그 항이 유의한지 본다.
1273     유의하면(p < alpha) 로짓이 직선이 아니라는 뜻이므로 변환·제곱항·구간화가 필요하다.
1274     이분형(더미) 변수는 두 점을 잇는 선이 언제나 직선이므로 검정 대상에서 제외한다.
1275
1276     Args:
1277         fit: `fit_model` 함수로 적합된 회귀분석 결과 객체.
1278         data: 회귀분석에 사용한 원본 데이터프레임. 독립변수와 종속변수를 모두 포함해야 한다.
1279         yname (str): 종속변수 컬럼명.
1280         xnames (list): 모델에 투입된 독립변수 이름 목록.
1281         targets (list): 검정할 변수 목록. None이면 `xnames` 중 연속형 전체 (기본값: None).
1282         alpha (float): 유의수준 (기본값: 0.05).
1283     """
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-2)

실습

□ 로짓 선형성 검정 함수 개선 (2/2)

```
1285    # --- 1) 검정 대상 결정 (상수항 제외, 연속형만) ---
1286    if yname is None:
1287        yname = fit.model.endog_names
1288
1289    if xnames is None:
1290        xnames = []
1291        for name in fit.model.exog_names:
1292            if name != "const":
1293                xnames.append(name)
1294
1295    if targets is None:
1296        targets = []
1297        for x in xnames:
1298            # 값의 종류가 3가지 이상인 변수만 연속형으로 간주한다
1299            if data[x].nunique() > 2:
1300                targets.append(x)
```

```
1298    # --- 2) Box-Tidwell 검정 (변수마다 x*ln(x) 항을 하나씩 추가) ---
1299                        ... 생략 ...
1300
1301    # 결과표 생성 및 출력
1302    result_df = DataFrame(rows).set_index("독립변수")
1303    #display(result_df)
1304    return result_df
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#09. 가정 점검

실습

1. 기준선 모형의 로짓 선형성 검정 (Box-Tidwell)

◎ 독립변수 x에 $x \times ln(x)$ 항을 추가해 **그 항이 유의한지** 본다.

○ 유의하면($p < 0.05$) 로짓이 직선이 아니라는 뜻 → **변환·제곱항·구간화가 필요**하다.

```python
my_logit.test_linear(fit_base, base)
```

Python

독립변수	z	p-value	linearity	위치이동	result
Size	4.323	0.000	False	True	대립가설 채택 → 로짓 선형성 위배(변환 필요)
Weight	11.161	0.000	False	True	대립가설 채택 → 로짓 선형성 위배(변환 필요)
Sweetness	0.605	0.545	True	True	귀무가설 채택 → 로짓 선형성 위배 근거 없음
Crunchiness	9.851	0.000	False	True	대립가설 채택 → 로짓 선형성 위배(변환 필요)
Juiciness	-8.075	0.000	False	True	대립가설 채택 → 로짓 선형성 위배(변환 필요)
Ripeness	-1.047	0.295	True	True	귀무가설 채택 → 로짓 선형성 위배 근거 없음
Acidity	-5.553	0.000	False	True	대립가설 채택 → 로짓 선형성 위배(변환 필요)

결과 — 7종 중 5종이 위배다. (Weight z = 11.16, Crunchiness z = 9.85, Juiciness z = −8.08, Acidity z = −5.55, Size z = 4.32)

- Sweetness(p = 0.545)와 Ripeness(p = 0.295)만 충족한다 → **위배가 가장 심한 Weight부터 처방한다.**

※ 이 변수들은 음수를 포함하므로 ln의 정의역을 맞추기 위해 **최솟값이 1이 되도록 평행이동**한 뒤 검정된다. (위치이동 = True)

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-3)

실습

□ 로짓 선형성 위배 처방 함수 정의

```
871 def fix_linear(data, y, alpha=0.05, allow_shifted_log=False, max_rounds=None, report=True):
872     """로짓 선형성 위배를 한 변수씩 처방하고 재검정하는 루프를 자동으로 수행한다.
873
874     Args:
875         data: 독립변수와 0/1 종속변수를 모두 포함한 데이터프레임.
876         y (str): 종속변수 컬럼명.
877         alpha (float): 유의수준 (기본값: 0.05).
878         allow_shifted_log (bool): 음수를 포함한 변수도 평행이동 후 로그 변환을 시도할지 여부.
879             이동량이 자의적이어서 계수 해석이 무너지므로 기본값은 False (기본값: False).
880         max_rounds (int): 최대 반복 횟수. None이면 연속형 독립변수 개수 (기본값: None).
881         report (bool): 라운드별 진행 상황과 처리 이력표를 출력할지 여부 (기본값: True).
882
883     Returns:
884         처방이 모두 반영된 회귀분석 결과 객체. 아래 속성이 함께 붙는다.
885             - `data_` (DataFrame): 파생변수가 반영된 데이터프레임
886             - `recipe_` (dict): 변수별 처방 내역. `apply_recipe` 에 그대로 넘겨 쓴다
887             - `history_` (DataFrame): 라운드별 처리 이력표
888             - `unresolved_` (list): 제곱항·로그변환으로도 해소되지 않은 변수 목록
889
890     Raises:
891         KeyError: 종속변수 컬럼이 `data` 에 없는 경우.
892         ValueError: alpha·max_rounds 가 유효하지 않은 경우.
893     """
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit.py (step-4)

□ 구현 과정 구성

```
871 def fix_linear(data, y, alpha=0.05, allow_shifted_log=False, max_rounds=None, report=True):
872     """로짓 선형성 위배를 한 변수씩 처방하고 재검정하는 루프를 자동으로 수행한다.…
894     # --- 1) 입력 검증 ---
895
896     # --- 2) 초기 상태 준비 ---
897
898     # --- 3) 진단 → 처방 → 검증 → 재진단 루프 ---
899
900     # --- 4) 결과 정리 ---
901
902
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-5)

실습

□ 입력 파라미터 검증

```
894     # --- 1) 입력 검증 ---
895     if y not in data.columns:
896         raise KeyError(f"종속변수 컬럼이 없습니다: '{y}'")
897
898     if not 0 < alpha < 1:
899         raise ValueError(f"alpha 는 0과 1 사이여야 합니다: {alpha}")
900
901     if max_rounds is not None and max_rounds < 1:
902         raise ValueError(f"max_rounds 는 1 이상이어야 합니다: {max_rounds}")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-6)

실습

□ 초기화

```
1177    # --- 2) 초기 상태 준비 ---
1178    tmp = data.copy()
1179    fit = fit_model(tmp, y=y)                # 기준선 모형
1180
1181    # 아직 처방하지 않은 연속형 독립변수 (이분형은 언제나 직선이므로 제외)
1182    pending = []
1183    for x in fit.model.exog_names:
1184        if x != "const" and tmp[x].nunique() > 2:
1185            pending.append(x)
1186
1187    if max_rounds is None:              # max_rounds 가 지정되지 않으면
1188        max_rounds = len(pending)       # 연속형 독립변수 개수만큼 반복한다
1189
1190    recipe = {}             # 확정된 처방
1191    unresolved = []         # 처방으로 해소하지 못한 변수
1192    history = []            # 라운드별 이력
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-5)

□ 처방 루프 구성

```
1194    # --- 3) 진단 → 처방 → 검증 → 재진단 루프 ---
1195    for rnd in range(1, max_rounds + 1):
1196        # --- 3-1) 진단: 아직 손대지 않은 변수만 다시 검정한다 ---
1197
1198        # --- 3-2) 위배가 가장 심한 변수 하나를 고른다 ---
1199
1200        # --- 3-3) 처방 ①: 중심화 후 제곱항 추가 ---
1201
1202        # --- 3-4) 처방 ②: 로그 변환 ---
1203
1204        # --- 3-5) 두 처방을 견주어 채택한다 ---
1205
1206        # --- 3-6) 둘 다 실패: 원본을 그대로 두고 미해소로 기록한다 ---
1207
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-6)

실습

□ 진단 루프 구현 (1/6)

```
1194    # --- 3) 진단 → 처방 → 검증 → 재진단 루프 ---
1195    for rnd in range(1, max_rounds + 1):
1196        # --- 3-1) 진단: 아직 손대지 않은 변수만 다시 검정한다 ---
1197        # 이미 처방한 변수를 다시 넣으면 짝이 되는 제곱항 때문에 판정이 무의미하다
1198        targets = [x for x in pending if x not in unresolved]
1199
1200        if not targets:
1201            break
1202
1203        bt = test_linear(fit, data=tmp, yname=y, targets=targets, alpha=alpha)
1204
1205        violated = bt[~bt["linearity"]]
1206
1207        if violated.empty:
1208            if report:
1209                print(f"[라운드 {rnd}] 위배 변수 없음 → 루프 종료")
1210            break
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit.py (step-7)

실습

□ 진단 루프 구현 (2/6)

```
1212        # --- 3-2) 위배가 가장 심한 변수 하나를 고른다 ---
1213        target = violated["z"].abs().idxmax()
1214
1215        if report:
1216            print(f"[라운드 {rnd}] 위배 {len(violated)}종 → "
1217                  f"'{target}' 처방 (z = {violated.loc[target, 'z']})")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit.py (step-8)

실습

□ 진단 루프 구현 (2/6)

```
1212        # --- 3-2) 위배가 가장 심한 변수 하나를 고른다 ---
1213        target = violated["z"].abs().idxmax()
1214
1215        if report:
1216            print(f"[라운드 {rnd}] 위배 {len(violated)}종 → "
1217                  f"'{target}' 처방 (z = {violated.loc[target, 'z']})")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-9)

□ 진단 루프 구현 (3/6)

```
1219        # --- 3-3) 처방 ①: 중심화 후 제곱항 추가 ---
1220        center = float(tmp[target].mean())
1221
1222        sq_data = tmp.copy()
1223        sq_data[target + "_c"] = sq_data[target] - center
1224        sq_data[target + "_c2"] = sq_data[target + "_c"] ** 2
1225        sq_data = sq_data.drop(columns=[target])
1226
1227        # 적합이 실패하면 제곱항은 후보에서 탈락시킨다
1228        try:
1229            sq_fit = fit_model(sq_data, y=y)
1230
1231            # 검증: 제곱항 자체가 유의한가 + 우도비 검정이 개선을 지지하는가
1232            sq_p = float(sq_fit.pvalues[target + "_c2"])
1233            lr_stat = 2 * (sq_fit.llf - fit.llf)
1234            lr_p = float(1 - chi2.cdf(lr_stat, 1))  # 늘어난 모수는 제곱항 1개
1235            sq_ok = bool(sq_p < alpha and lr_p < alpha)
1236        except Exception:
1237            sq_fit, sq_p, lr_p, sq_ok = None, np.nan, np.nan, False
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-10)

실습

□ 진단 루프 구현 (4/6)

```
1066        # --- 3-4) 처방 ②: 로그 변환 ---
1067        # 최솟값이 0 이하이면 ln 정의역을 맞추기 위해 평행이동해야 하는데,
1068        # 이동량이 자의적이어서 계수 해석이 무너지므로 기본값에서는 시도하지 않는다
1069        min_value = float(tmp[target].min())
1070        shift = 0.0 if min_value > 0 else -min_value + 1.0
1071
1072        log_fit, log_ok = None, False
1073
1074        if min_value > 0 or allow_shifted_log:
1075            log_data = tmp.copy()
1076            log_data[target + "_log"] = np.log(log_data[target] + shift)
1077            log_data = log_data.drop(columns=[target])
1078
1079            try:
1080                log_fit = fit_model(log_data, y=y)
1081
1082                # 검증: 로그 변환은 모수가 늘지 않으므로 우도비 검정을 쓸 수 없다.
1083                # 대신 AIC가 좋아졌고 그 변수의 선형성이 회복되었는지로 판정한다
1084                aic_better = bool(log_fit.aic < fit.aic)
1085
1086                log_bt = test_linear(log_fit, data=log_data, yname=y,
1087                                     targets=[target + "_log"], alpha=alpha)
1088                log_linear = bool(log_bt.loc[target + "_log", "linearity"])
1089                log_ok = bool(aic_better and log_linear)
1090            except Exception:
1091                log_fit, log_ok = None, False
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-11)

실습

□ 진단 루프 구현 (5/6 - 1)

```
1266        # --- 3-5) 두 처방을 견주어 채택한다 ---
1267        # 제곱항은 단조 곡선도 근사하므로, '유의하지 않을 때만 로그'로 두면 로그가 맞는 경우에도 제곱항이
            먼저 채택된다.
1268        # 따라서 둘 다 검증을 통과하면 AIC가 낮은 쪽을 고른다
1269        if sq_ok and log_ok: method = "square" if sq_fit.aic <= log_fit.aic else "log"
1270        elif sq_ok:          method = "square"
1271        elif log_ok:         method = "log"
1272        else:                method = None
1273
1274        if method == "square":…
1288
1289                                    ...next...
1290
1291        elif method == "log":…
1306
1307                                    ...next...
1308
1309        # --- 3-6) 둘 다 실패: 원본을 그대로 두고 미해소로 기록한다 ---
```



/helpers/my_logit.py (step-12)

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측

□ 진단 루프 구현 (5/6 - 2)

```
1275        if method == "square":
1276            tmp, fit = sq_data, sq_fit
1277            recipe[target] = {"method": "square", "center": center, "shift": None}
1278            pending.remove(target)
1279
1280            history.append({
1281                "라운드": rnd, "변수": target, "처방": "제곱항",
1282                "제곱항 p": round(sq_p, 4), "검증 p": round(lr_p, 4),
1283                "AIC": round(sq_fit.aic, 2), "채택": True,
1284            })
1285
1286            if report:
1287                print(f"    → 제곱항 채택 (제곱항 p = {sq_p:.4f}, "
1288                      f"우도비 p = {lr_p:.4e}, AIC = {sq_fit.aic:.2f})")
1289
1290        elif method == "log":…
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-13)

실습

□ 진단 루프 구현 (5/6 - 3)

```
1275        if method == "square":…
1289
1290        elif method == "log":
1291            tmp, fit = log_data, log_fit
1292            recipe[target] = {"method": "log", "center": None, "shift": shift}
1293            pending.remove(target)
1294
1295            history.append({
1296                "라운드": rnd, "변수": target, "처방": "로그변환",
1297                "제곱항 p": round(sq_p, 4) if np.isfinite(sq_p) else np.nan,
1298                "검증 p": None,
1299                "AIC": round(log_fit.aic, 2), "채택": True,
1300            })
1301
1302            if report:
1303                reason = "제곱항 기각" if not sq_ok else "AIC 우위"
1304                print(f"    → 로그변환 채택 ({reason}, AIC = {log_fit.aic:.2f})")
1305
1306        # --- 3-6) 둘 다 실패: 원본을 그대로 두고 미해소로 기록한다 ---
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-14)

실습

□ 진단 루프 구현 (6/6)

```
1306        # --- 3-6) 둘 다 실패: 원본을 그대로 두고 미해소로 기록한다 ---
1307        else:
1308            unresolved.append(target)
1309
1310            history.append({
1311                "라운드": rnd, "변수": target, "처방": "미해소",
1312                "제곱항 p": round(sq_p, 4) if np.isfinite(sq_p) else np.nan,
1313                "검증 p": None,
1314                "AIC": round(fit.aic, 2), "채택": False,
1315            })
1316
1317            if report:
1318                print(f"    → 제곱항·로그변환 모두 기각 → 해결되지 않음")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-15)

□ 결과 정리

```
1320    # --- 4) 결과 정리 ---
1321    history_df = DataFrame(history)
1322
1323    if report:
1324        print()
1325        if not history_df.empty:
1326            display(history_df.set_index("라운드"))
1327
1328        if unresolved:
1329            print(f"⚠ 미해소 변수: {unresolved}\n"
1330                  f"  제곱항·로그변환으로 담기지 않는 형태이므로 "
1331                  f"구간화나 스플라인이 필요하다는 신호입니다. 한계점으로 보고하세요.")
1332
1333    fit.data_ = tmp
1334    fit.recipe_ = recipe
1335    fit.history_ = history_df
1336    fit.unresolved_ = unresolved
1337
1338    return fit
```



#09. 가정 검정

2. 진단 → 처방 → 검증 → 재진단 루프

```
1 fit_fixed = my_logit.fix_linear(base, y=target)
```

✓ 마지막 실행: 오전 7:09:52, 기간: 0.1s Python

```
[라운드 1] 위배 5종 → 'Weight' 처방 (z = 11.1613)
    → 제곱항 채택 (제곱항 p = 0.0000, 우도비 p = 0.0000e+00, AIC = 3962.44)
[라운드 2] 위배 5종 → 'Crunchiness' 처방 (z = 9.0855)
    → 제곱항 채택 (제곱항 p = 0.0000, 우도비 p = 0.0000e+00, AIC = 3867.27)
[라운드 3] 위배 3종 → 'Juiciness' 처방 (z = -10.5276)
    → 제곱항 채택 (제곱항 p = 0.0000, 우도비 p = 0.0000e+00, AIC = 3744.33)
[라운드 4] 위배 3종 → 'Acidity' 처방 (z = -7.4836)
    → 제곱항 채택 (제곱항 p = 0.0000, 우도비 p = 3.2307e-14, AIC = 3688.74)
[라운드 5] 위배 2종 → 'Ripeness' 처방 (z = -3.0869)
    → 제곱항 채택 (제곱항 p = 0.0023, 우도비 p = 2.1255e-03, AIC = 3681.30)
[라운드 6] 위배 1종 → 'Size' 처방 (z = 2.9542)
    → 제곱항 채택 (제곱항 p = 0.0025, 우도비 p = 2.2468e-03, AIC = 3673.97)
[라운드 7] 위배 변수 없음 → 루프 종료
```

라운드	변수	처방	제곱항 p	검증 p	AIC	채택
1	Weight	제곱항	0.000	0.000	3962.440	True
2	Crunchiness	제곱항	0.000	0.000	3867.270	True
3	Juiciness	제곱항	0.000	0.000	3744.330	True
4	Acidity	제곱항	0.000	0.000	3688.740	True
5	Ripeness	제곱항	0.002	0.002	3681.300	True
6	Size	제곱항	0.003	0.002	3673.970	True

인사이트 — 하나씩 처방하니 Crunchiness도 해소됐다

- 6개 라운드에 걸쳐 **Weight → Crunchiness → Juiciness → Acidity → Ripeness → Size** 순으로 위배가 심한 변수부터 하나씩 처방됐고, **7라운드에서 위배 변수가 사라져 루프가 종료**됐다.
- Sweetness는 끝까지 위배로 잡히지 않아 **원본 그대로 남았다.**
- 위배 변수 개수는 5 → 5 → 3 → 3 → 2 → 1 → 0으로 줄지만 **한 번에 하나씩 줄지 않는다.** 한 변수를 고치면 다른 변수의 진단이 함께 바뀌기 때문 — **재진단을 반드시 해야 하는 이유**
- Crunchiness는 T검정 p = 0.985(완전 무관), 기준선 모형 p = 0.253(제거 후보)였는데 로짓 선형성 진단에서는 **두 번째로 심한 위배**로 잡힌다.
  - 양 극단이 모두 good 쪽이라 평균 비교로는 상쇄되어 보이지 않았을 뿐이다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-15)

실습

□ 독립성 검정 함수 정의

```
1301 def test_independent(fit):
1302     """Durbin-Watson으로 잔차의 독립성을 검정한다.
1303
1304     로지스틱 회귀는 잔차가 0/1 구조라 일반 잔차를 쓸 수 없으므로 **피어슨 잔차**를 사용한다.
1305     본래 시계열 전용 검정이므로, 시간 순서가 없는 데이터에서는 통계량이 정상이어도
1306     실제로는 독립이 아닐 수 있다. 최종 판단은 **자료 수집 설계**로 해야 한다.
1307
1308     Args:
1309         fit: `fit_model` 함수로 적합된 회귀분석 결과 객체.
1310
1311     Returns:
1312         DataFrame: Durbin-Watson 통계량과 독립성 판정을 담은 단일 행 결과표.
1313     """
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-16)

실습

□ 독립성 검정 함수 구현

```
1301 def test_independent(fit):
1302     """Durbin-Watson으로 잔차의 독립성을 검정한다.…
1314     # --- 1) Durbin-Watson 통계량 계산 (피어슨 잔차 사용) ---
1315     dw = float(durbin_watson(fit.resid_pearson))
1316
1317     # --- 2) DW값에 따른 독립성 판정 및 해석 ---
1318     if 1.5 <= dw <= 2.5:
1319         independence = True
1320         conclusion = "독립성 만족"
1321     elif dw < 1.5:
1322         independence = False
1323         conclusion = "독립성 위반 (양(+)의 자기상관)"
1324     else:
1325         independence = False
1326         conclusion = "독립성 위반 (음(-)의 자기상관)"
1327
1328     # --- 3) 단일 행 결과표 구성 및 출력 ---
1329     result_df = DataFrame([{
1330             "statistic": round(dw, 4),
1331             "independence": independence,
1332             "result": conclusion,
1333         }],
1334         index=["Durbin-Watson"])
1335
1336     return result_df
```



#09. 가정 검정

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측

4. 관측치의 독립성 - Durbin Watson

```
1 my_logit.test_independent(fit_fixed)
```

✓ 마지막 실행: 오전 7:13:47, 기간: 0.0s Python

	statistic	independence	result
Durbin-Watson	2.022	True	독립성 만족

결과 — DW = **2.02**로 기준 범위(1.5 ~ 2.5)의 한가운데다. → **독립성 만족.**
사과 4,000개는 서로 다른 개체이고 수집 순서에 의미가 없으므로 설계상으로도 독립이 타당하다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#09. 가정 검정

실습

5. 가정 검정 결과 확정

◎ 처방 대상 변수 목록을 확정한다. 이 목록이 **다음 단계 파이프라인의 옵션**이 된다.

```python
# 확정된 처방 내역 (변수별 처방 방식과 기준값)
recipe = fit_fixed.recipe_

for col, rule in recipe.items():
    print(f"{col:14s} : {rule['method']}")

extra_cols = [c for c in continuous_cols if c not in recipe]
print("\n처방 제외 (선형성 충족) :", extra_cols)
print("미해소 (구간화·스플라인 필요) :", fit_fixed.unresolved_)
```

✓ 마지막 실행: 오전 7:16:35, 기간: 0.0s Python

```
Weight         : square
Crunchiness    : square
Juiciness      : square
Acidity        : square
Ripeness       : square
Size           : square

처방 제외 (선형성 충족) : ['Sweetness']
미해소 (구간화·스플라인 필요) : []
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



연습문제 사과 품질 분류 데이터셋 활용 로지스틱 회귀 #04

□ 가정 확인 절차를 수행하고 다음 물음에 답하시오.

21. 종속변수 Quality를 0/1로 라벨링했을 때, good에 부여되는 숫자는 무엇인가요? (정수 1개)
22. 7개 변수를 모두 넣은 기준선 모형에서, 유의수준 5% 기준으로 유의하지 않은 독립변수는 몇 개인가요? (정수 1개)
23. 이변량 분석(T검정)에서 유의확률 0.797로 "차이가 없다"고 나왔지만, 기준선 모형에서는 z = −13.70으로 강하게 유의해진 변수는 무엇인가요? (변수명 1개)
24. 문제 23처럼 단독 비교에서는 효과가 가려져 있다가 다른 변수를 통제하자 드러나는 현상을 무엇이라 하나요? (용어 1개)
25. 기준선 모형에서 표준화 계수(βstd)가 가장 큰 변수는 무엇인가요? (변수명 1개)
26. Box-Tidwell 검정 결과, 7종 중 로짓 선형성을 위배한 변수는 몇 종인가요? (정수 1개)
27. 위배 정도가 가장 심한(검정통계량 z의 절댓값이 가장 큰) 변수는 무엇인가요? (변수명 1개)
28. 7종 중 선형성을 충족해 처방 대상에서 제외된 변수는 무엇인가요? (변수명 1개)
29. Weight를 처방한 다음 라운드에서 다시 진단했을 때, 위배로 잡힌 변수는 몇 종인가요? (정수 1개)
30. 루프가 종료되기까지 실제로 처방이 이루어진 라운드는 몇 번인가요? (정수 1개)
31. 6종에 공통으로 채택된 처방 방법은 무엇인가요? (영문 1개)
32. 제곱항과 함께 후보였던 로그 변환이 이 데이터에서는 한 번도 시도되지 않았습니다. 7종 모두가 포함하고 있어 로그를 쓸 수 없게 만든 값은 무엇인가요? (용어 1개)
33. 제곱항으로도 로그 변환으로도 해소되지 않아 구간화·스플라인이 필요하다고 기록된 변수는 몇 개인가요? (정수 1개)
34. Durbin-Watson 통계량은 얼마인가요? (소수 셋째 자리)
35. 제곱항이 모형에 있으면 1차항이 유의하지 않아도 남겨야 한다는 원칙을 무엇이라 하나요? (용어 1개)

ITWILL 이광호강사

5. 분석 파이프라인

모든 실습이 하나의 노트북 파일에서 이어서 진행됩니다.

실습량이 매우매우매우매우 많습니다.

ITWILL 이광호 강사

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



단계 7-1: 파이프라인 함수 설계

□ 전처리 옵션을 함수 하나에 스위치로 모아 둔다. (함수 내부의 실행 순서가 중요)

스위치로 모아 두는 옵션

이상치 대체	로짓 선형성 처방	다중공선성 제거	정규화	후진소거

함수 내부 실행 순서

STEP 1
이상치 대체
원본 변수에 먼저

▶

STEP 2
처방 재현
(파생변수 생성)

▶

STEP 3
다중공선성 제거

▶

STEP 4
정규화

▶

STEP 5
모델 적합
후진소거 포함

STEP 1과 2의 순서를 어기면
파생변수에 IQR 경계를 적용하면 곡선의 모양이 잘려 나간다.
따라서 이상치 대체는 반드시 **원본 변수에** 먼저 적용하고, 그 뒤에 중심화·제곱항을 만든다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-17)

실습

□ 로지스틱 회귀 파이프라인 함수 정의 (1/2)

```
1207 def fit_pipeline(data, y, columns=None, *,
1208                  # --- 1) 이상치 대체 (IQR 경계값, 행 삭제 없음) ---
1209                  outlier=False,              # 이상치 대체 수행 여부
1210                  # --- 2) 로짓 선형성 처방 재현 ---
1211                  recipe=None,                # fix_linear 이 확정한 처방 내역
1212                  # --- 3) 더미변수 인코딩 ---
1213                  encode=True,                # 명목형 독립변수의 더미 인코딩 수행 여부
1214                  # --- 4) 다중공선성 제거 (VIF) ---
1215                  vif=False,                  # 다중공선성 제거 수행 여부
1216                  vif_threshold=10.0,         # VIF 임계값
1217                  # --- 5) 정규화 ---
1218                  scale=False,                # 정규화 수행 여부
1219                  scale_method="standard",    # 사용할 스케일러 이름
1220                  # --- 6) 모델 적합 ---
1221                  backward=False,             # 후진소거법 수행 여부
1222                  alpha=0.05,                 # 후진소거법의 변수 제거 기준 유의수준
1223                  # --- 기타 ---
1224                  name=None,                  # 모델을 구분할 이름. 결과 객체의 `name_` 속성이 된다
1225                  verbose=False):             # 단계별 전처리 내역 출력 여부
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-18)

실습

□ 로지스틱 회귀 파이프라인 함수 정의 (2/2)

```
1207 def fit_pipeline(data, y, columns=None, *, ... ):
1208     """플래그로 지정한 전처리를 수행한 뒤 로지스틱 회귀모델을 적합한다.
1209
1210     Args:
1211         data (DataFrame): 독립변수와 0/1 종속변수를 모두 포함하는 데이터프레임.
1212         y (str): 종속변수로 사용할 컬럼명.
1213         columns (list): 이상치 대체 대상이 되는 원본 연속형 독립변수 목록.
1214         outlier (bool): 이상치를 IQR 경계값으로 대체할지 여부 (기본값: False).
1215         recipe (dict): `fix_linear` 이 돌려준 `recipe_`. None이면 처방을 적용하지 않는다 (기본값: None).
1216         encode (bool): 명목형 독립변수의 더미 인코딩 수행 여부 (기본값: True).
1217         vif (bool): 다중공선성 제거 수행 여부 (기본값: False).
1218         vif_threshold (float): VIF 임계값 (기본값: 10.0).
1219         scale (bool): 정규화 수행 여부 (기본값: False).
1220         scale_method (str): 사용할 스케일러 이름 (기본값: 'standard').
1221         backward (bool): 후진소거법 수행 여부 (기본값: False).
1222         alpha (float): 후진소거법의 변수 제거 기준 유의수준 (기본값: 0.05).
1223         name (str): 모델을 구분할 이름. 결과 객체의 `name_` 속성이 된다 (기본값: None).
1224         verbose (bool): 단계별 전처리 내역 출력 여부 (기본값: False).
1225
1226     Returns:
1227         적합이 완료된 회귀분석 결과 객체. 아래 속성이 함께 붙는다.
1228             - `data_` (DataFrame): 전처리가 끝난 데이터. `report_variables` 등이 사용한다
1229             - `recipe_` (dict): 적용한 로짓 선형성 처방. 신규 예측 시 `apply_recipe` 에 넘긴다
1230             - `name_` (str): 모델을 구분할 이름
1231
1232     Raises:
1233         KeyError: 종속변수 컬럼이 `data` 에 없는 경우.
1234         ValueError: 결측치가 남아 있는 경우.
1235     """
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-19)

실습

□ 구현 과정 구성

```
1207 def fit_pipeline(data, y, columns=None, ... ):
1208     """플래그로 지정한 전처리를 수행한 뒤 로지스틱 회귀모델을 적합한다.…
1237     # --- 1) 종속변수 확인 및 작업본 준비 ---
1238
1239     # --- 2) 이상치 대체 대상 확정 ---
1240
1241     # --- 3) 이상치 대체 (IQR 경계값, 행 삭제 없음) ---
1242
1243     # --- 4) 로짓 선형성 처방 재현 (중심화+제곱항 또는 로그변환) ---
1244
1245     # --- 5) 더미변수 인코딩 (명목형 독립변수가 있을 때만 동작한다) ---
1246
1247     # --- 6) 다중공선성 제거 (VIF) ---
1248
1249     # --- 7) 정규화 ---
1250
1251     # --- 8) 결측치 점검 ---
1252
1253     # --- 9) 모델 적합 (backward=True 이면 후진소거법 수행) ---
1254
1255     # --- 10) 보고에 필요한 정보를 결과 객체에 붙여 반환 ---
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-20)

실습

□ 종속변수 확인 및 작업본 준비 / 이상치 대체 대상 확정

```
1255    # --- 1) 종속변수 확인 및 작업본 준비 ---
1256    if y not in data.columns:
1257        raise KeyError(f"종속변수 '{y}'가 데이터프레임의 컬럼에 존재하지 않습니다.")
1258
1259    df = data.copy()    # 원본을 보존하기 위해 복사본으로 작업
1260
1261    # --- 2) 이상치 대체 대상 확정 ---
1262    # 지정이 없으면 숫자형 독립변수를 자동 선택한다
1263    if columns is None:
1264        columns = []
1265        for c in df.select_dtypes(include="number").columns:
1266            if c != y:
1267                columns.append(c)
1268    else:
1269        missing = []
1270        for c in columns:
1271            if c not in df.columns:
1272                missing.append(c)
1273
1274        if missing:
1275            raise KeyError(f"데이터프레임에 존재하지 않는 컬럼입니다: {missing}")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-21)

실습

□ 이상치 대체 / 로짓 선형성 처방 재현 / 더미변수 인코딩

```
1277    # --- 3) 이상치 대체 (IQR 경계값, 행 삭제 없음) ---
1278    # -> 파생변수를 만들기 전에 원본 변수에 적용해야 곡선이 잘리지 않는다
1279    if outlier and columns:
1280        df = my_prep.replace_outlier(df, columns=columns, verbose=verbose)
1281
1282    # --- 4) 로짓 선형성 처방 재현 (중심화+제곱항 또는 로그변환) ---
1283    if recipe:
1284        df = apply_recipe(df, recipe)
1285
1286    # --- 5) 더미변수 인코딩 (명목형 독립변수가 있을 때만 동작한다) ---
1287    if encode:
1288        nominal_cols = []
1289        for c in df.select_dtypes(include=["category", "object"]).columns:
1290            if c != y:
1291                nominal_cols.append(c)
1292
1293        if nominal_cols:
1294            df = my_prep.dummies(df, columns=nominal_cols, verbose=verbose)
1295
1296    # 현재 시점의 독립변수 목록 (파생변수·더미가 추가되었을 수 있다)
1297    alive = [c for c in df.columns if c != y]
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-22)

실습

□ 다중공선성 제거 / 정규화 / 결측치 점검

```
1299    # --- 6) 다중공선성 제거 (VIF) ---
1300    if vif:
1301        df = my_prep.reduce_vif(df, columns=alive, threshold=vif_threshold,
1302                                verbose=verbose)
1303        alive = [c for c in df.columns if c != y]
1304
1305    # --- 7) 정규화 ---
1306    if scale:
1307        df = my_prep.scaling(df, columns=alive, method=scale_method, verbose=verbose)
1308
1309    # --- 8) 결측치 점검 ---
1310    na_cols = list(df.columns[df.isna().any()])
1311
1312    if na_cols:
1313        raise ValueError(f"결측치가 있는 컬럼이 있습니다: {na_cols}\n"
1314                         f"데이터 품질 점검 단계에서 먼저 처리하세요.")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit.py (step-23)

실습

□ 모델 적합 / 결과 보고

```
1316    # --- 9) 모델 적합 (backward=True 이면 후진소거법 수행) ---
1317    fit = auto_logit(df, y=y, backward=backward, alpha=alpha,
1318                     report=False, plot=False)
1319
1320    # --- 10) 보고에 필요한 정보를 결과 객체에 붙여 반환 ---
1321    fit.data_ = df                          # report_variables 등이 β·VIF 계산에 사용한다
1322    fit.recipe_ = recipe or {}              # 신규 예측 시 apply_recipe 에 그대로 넘긴다
1323    fit.name_ = name                        # compare_models 가 채워 주기도 한다
1324
1325    return fit
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



단계 7-2: 체크포인트 비교 설계

□ 전처리를 하나씩 누적한 조합을 정의하고, 반복문으로 일괄 적합한 뒤 비교한다.

STEP 1
누적 조합을 딕셔너리로 정의 ▶

STEP 2
반복문으로 일괄 적합 ▶

STEP 3
성능 비교표 생성

설계 규칙 — 모든 조합에서 공선성 제거는 기본으로 적용한다. / 두 계열을 동일한 조합으로 맞춘다.

계열 A: 가정 검정 미반영	기준선	+ 후진소거	+ 이상치 대체	+ 정규화
계열 B: 가정 검정 반영	기준선	+ 후진소거	+ 이상치 대체	+ 정규화

두 계열을 같은 조합으로 맞추는 이유

그래야 **가정 처방의 순수한 효과만 떼어 볼 수 있다** — 다른 조건이 같아야 차이의 원인을 특정할 수 있다.

비교표를 읽는 규칙

1순위 지표로 정렬하되, **근소 격차 그룹 안에서는 변수가 적은 쪽**을 후보로 삼는다.

자동으로 지목된 1순위가 최종 답이 아니다

제거된 변수가 **계층 원칙을 깨뜨리지 않았는지 분석자가 검토**하고, 필요하면 **뒤집는다**.

ITWILL 이광호강사

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#10. 로지스틱 회귀 파이프라인

1. 전처리 체크포인트 정의

◎ 전처리 단계를 하나씩 누적한 체크포인트 (아래로 갈수록 전처리가 하나 더 붙는다)

○ **0~3번은 가정 검정을 반영하지 않은 조합**, **4~7번은 recipe를 적용한 조합**이다.

◎ **두 계열의 전처리 조합을 동일**하게 맞춰 로짓 선형성 처방의 순수한 효과를 볼 수 있게 한다.

◎ **공선성 제거(vif=True)는 모든 체크포인트에 공통**으로 걸어 둔다.

○ 앞 과정에서 이미 제거 대상이 없음을 확인했으므로, 별도의 비교 단계로 쪼갤 이유가 없다.

```python
checkpoints = {
    # --- 가정 검정 미반영 ---
    "0_기준선":              dict(vif=True),
    "1_후진소거":            dict(backward=True, vif=True),
    "2_이상치대체":          dict(backward=True, outlier=True, vif=True),
    "3_정규화":              dict(backward=True, outlier=True, vif=True, scale=True),

    # --- 가정 검정 반영 (로짓 선형성 처방) ---
    "4_가정반영":            dict(recipe=recipe, vif=True),
    "5_가정반영+후진소거":   dict(recipe=recipe, backward=True, vif=True),
    "6_가정반영+이상치대체": dict(recipe=recipe, backward=True, outlier=True, vif=True),
    "7_가정반영+정규화":     dict(recipe=recipe, backward=True, outlier=True, vif=True, scale=True),
}
```

✓ 마지막 실행: 오전 7:32:14, 기간: 0.0s Python

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#10. 로지스틱 회귀 파이프라인

실습

2. 모델별 체크 포인트 적합

```python
fits = {}    # 적합된 모델을 누적할 딕셔너리

for name in checkpoints:
    print(f"\n▶ --- {name} ---")

    fits[name] = my_logit.fit_pipeline(
        base, target, continuous_cols, name=name, **checkpoints[name])

print("\n누적된 모델 :", len(fits), "개")
```

✓ 마지막 실행: 오전 7:32:14, 기간: 0.3s Python

```
▶ --- 0_기준선 ---

완료! 남은 변수: ['Acidity', 'Crunchiness', 'Juiciness', 'Ripeness', 'Size', 'Sweetness', 'Weight']
최대 VIF = 1.48


▶ --- 1_후진소거 ---

완료! 남은 변수: ['Acidity', 'Crunchiness', 'Juiciness', 'Ripeness', 'Size', 'Sweetness', 'Weight']
최대 VIF = 1.48

유의하지 않은 독립변수 제거 → Crunchiness (p = 0.2532)

▶ --- 2_이상치대체 ---

완료! 남은 변수: ['Acidity', 'Crunchiness', 'Juiciness', 'Ripeness', 'Size', 'Sweetness', 'Weight']
```

… 출력 생략 …

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-24)

실습

□ 모델 성능 비교 함수 정의

```
1190 def compare_models(fits, metric="AUC", sub_metric="변수수", tolerance=0.05,
1191                    threshold=0.5, digits=4, report=True):
1192     """여러 로지스틱 모델의 분류 성능을 한 표로 정리해 좋은 순으로 정렬하고, 최고 모델을 반환한다.
1193     주 지표 1위와의 격차가 tolerance 이내면 '근소 격차 그룹'으로 묶어 보조 지표로 순서를 정한다.
1194
1195     `my_ols.compare_models` 와 같은 방식으로 동작하되, 회귀 지표(RMSE·R²) 대신
1196     분류 지표(정확도·F1·AUC)와 모형 적합도 지표(Pseudo R²·AIC)로 비교한다.
1197
1198     Args:
1199         fits (dict): {모델이름: 적합된 회귀분석 결과 객체} 형태의 딕셔너리.
1200         metric (str): 정렬 기준이 되는 주 성능평가지표 (기본값: 'AUC').
1201         sub_metric (str): 근소 격차 그룹 안에서 적용할 보조 지표. None이면 미사용 (기본값: '변수수').
1202         tolerance (float): 근소 격차로 판단할 주 지표의 상대격차. 0이면 순수 크기 비교 (기본값: 0.05).
1203         threshold (float): 확률을 0/1로 분류하는 임계값 (기본값: 0.5).
1204         digits (int): 표에 표시할 소수점 자릿수 (기본값: 4).
1205         report (bool): 성능 비교표를 화면에 출력할지 여부 (기본값: True).
1206
1207     Returns:
1208         성능이 가장 좋은 모델의 회귀분석 결과 객체(표의 첫 행). 아래 속성이 함께 붙는다.
1209             - `name_` (str): 모델 이름. `fits` 의 키에서 채워진다
1210             - `score_table_` (DataFrame): 모델명을 인덱스로 하는 성능 비교표.
1211               성능이 좋은 모델이 위에 오며, 맨 끝에 1위 대비 상대격차인 `Gap(%)` 컬럼이 붙는다
1212
1213     Raises:
1214         TypeError: `fits` 가 딕셔너리가 아니거나 값이 회귀분석 결과 객체가 아닌 경우.
1215         ValueError: `fits` 가 비었거나 지표 이름·tolerance·threshold 가 유효하지 않은 경우.
1216     """
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit (step-25)

실습

□ 구현 과정 설계

```
1190 def compare_models(fits, metric="AUC", sub_metric="변수수", tolerance=0.05,
1191                    threshold=0.5, digits=4, report=True):
1192     """여러 로지스틱 모델의 분류 성능을 한 표로 정리해 좋은 순으로 정렬하고, 최고 모델을 반환한다.…
1217     # --- 1) 지표별 '성능이 좋은 방향' 정의 (True = 값이 클수록 좋음) ---
1218
1219     # --- 2) 입력 검증 ---
1220
1221     # --- 3) 모델별 성능지표 계산 ---
1222
1223     # --- 4) 1위 대비 주 지표의 상대격차 계산 ---
1224
1225     # --- 5) 근소 격차 그룹을 먼저 정렬하고 나머지를 뒤에 붙인다 ---
1226
1227     # --- 6) 성능표 출력 ---
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit (step-26)

실습

□ 지표별 부호 정리

```
1217    # --- 1) 지표별 '성능이 좋은 방향' 정의 (True = 값이 클수록 좋음) ---
1218    metrics = {
1219        "변수수": False,          # 같은 성능이라면 변수가 적은 모델이 간명하다
1220        "정확도": True,
1221        "정밀도": True,
1222        "재현율": True,
1223        "F1": True,
1224        "AUC": True,
1225        "Pseudo R²": True,
1226        "AIC": False,
1227    }
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-27)

실습

□ 입력 파라미터 검증

```
1229    # --- 2) 입력 검증 ---
1230    if not isinstance(fits, dict):
1231        raise TypeError(f"fits 는 딕셔너리여야 합니다: {type(fits).__name__}")
1232
1233    if not fits:
1234        raise ValueError("비교할 모델이 없습니다.")
1235
1236    for name, fit in fits.items():
1237        if not hasattr(fit, "prsquared"):
1238            raise TypeError(f"'{name}' 의 값이 로지스틱 회귀분석 결과 객체가 아닙니다: "
1239                            f"{type(fit).__name__}")
1240
1241    for m in [metric, sub_metric]:
1242        if m is not None and m not in metrics:
1243            raise ValueError(f"지원하지 않는 지표입니다: '{m}' "
1244                             f"(사용 가능: {list(metrics.keys())})")
1245
1246    if tolerance < 0:
1247        raise ValueError(f"tolerance 는 0 이상이어야 합니다: {tolerance}")
1248
1249    if not 0 < threshold < 1:
1250        raise ValueError(f"threshold 는 0과 1 사이여야 합니다: {threshold}")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-28)

실습

□ 성능지표 계산

```
1252    # --- 3) 모델별 성능지표 계산 ---
1253    result = []
1254
1255    for name, fit in fits.items():
1256        y_true = np.asarray(fit.model.endog).astype(int)    # 실제 종속변수(0/1)
1257        proba = np.asarray(fit.predict())                   # 1이 될 확률
1258        y_pred = (proba > threshold).astype(int)            # 임계값 기준 예측 범주
1259
1260        result.append({
1261            "모델": name,
1262            "변수수": int(fit.df_model),          # 상수항을 제외한 독립변수 개수
1263            "정확도": accuracy_score(y_true, y_pred),
1264            "정밀도": precision_score(y_true, y_pred, zero_division=0),
1265            "재현율": recall_score(y_true, y_pred, zero_division=0),
1266            "F1": f1_score(y_true, y_pred, zero_division=0),
1267            "AUC": roc_auc_score(y_true, proba),
1268            "Pseudo R²": fit.prsquared,
1269            "AIC": fit.aic,
1270        })
1271
1272    rdf = DataFrame(result).set_index("모델")
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-29)

실습

□ 상대격차 계산

```
1274    # --- 4) 1위 대비 주 지표의 상대격차 계산 ---
1275    # 지표마다 좋은 방향이 다르므로 metrics 에 기록해 둔 방향으로 최적값을 찾는다
1276    higher_is_better = metrics[metric]
1277
1278    if higher_is_better:
1279        best = rdf[metric].max()
1280        diff = best - rdf[metric]       # 클수록 좋은 지표는 1위보다 작을수록 나쁘다
1281    else:
1282        best = rdf[metric].min()
1283        diff = rdf[metric] - best       # 작을수록 좋은 지표는 1위보다 클수록 나쁘다
1284
1285    # AIC 처럼 값이 음수인 지표도 있으므로 최적값의 절댓값을 분모로 삼는다.
1286    # 최적값이 0 이면 나눌 수 없으므로 격차를 값의 차이 그대로 본다
1287    if best != 0:
1288        denominator = abs(best)
1289    else:
1290        denominator = 1.0
1291
1292    rdf["Gap(%)"] = (diff / denominator * 100).round(2)    # 양수일수록 1위보다 나쁨
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-30)

□ 성능순 정렬

```
1294    # --- 5) 근소 격차 그룹을 먼저 정렬하고 나머지를 뒤에 붙인다 ---
1295    # 주 지표가 사실상 비슷한(격차가 tolerance 이내인) 모델끼리는 보조 지표로 순서를 정한다
1296    close = rdf["Gap(%)"] <= tolerance * 100
1297
1298    by = [metric]
1299    ascending = [not higher_is_better]
1300
1301    if sub_metric:
1302        # 보조 지표를 앞에 두어야 근소 격차 그룹 안에서 우선 적용된다
1303        by.insert(0, sub_metric)
1304        ascending.insert(0, not metrics[sub_metric])
1305
1306    front = rdf[close].sort_values(by=by, ascending=ascending)
1307    back = rdf[~close].sort_values(by=[metric], ascending=[not higher_is_better])
1308
1309    score_table = concat([front, back]).round(digits)
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit (step-31)

실습

□ 성능표 출력

```
1311    # --- 6) 성능표 출력 ---
1312    if report:
1313        display(score_table)
1314
1315    # 각 모델에 딕셔너리 키를 이름으로 새겨 둔다 (직접 지정한 name_ 이 없을 때만)
1316    for model_name, fit in fits.items():
1317        if getattr(fit, "name_", None) is None:
1318            fit.name_ = model_name
1319
1320    # 표는 성능순으로 정렬되어 있으므로 첫 행이 곧 최고 모델이다
1321    best = fits[score_table.index[0]]
1322    best.score_table_ = score_table
1323
1324    return best
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#10. 로지스틱 회귀 파이프라인

실습

□ 전체 모델 성능 비교

◎ 성능을 바꾼 것은 전처리가 아니라 가정 검정이었다.

```
1 best = my_logit.compare_models(fits)
2 print("간명성 기준 1순위 :", best.name_)
```

✓ 마지막 실행: 오전 7:32:14, 기간: 0.0s Python

모델	변수수	정확도	정밀도	재현율	F1	AUC	Pseudo R^2	AIC	Gap(%)
5_가정반영+후진소거	12	0.791	0.784	0.804	0.794	0.868	0.343	3672.041	0.000
6_가정반영+이상치대체	12	0.793	0.787	0.805	0.796	0.868	0.341	3677.811	0.060
7_가정반영+정규화	12	0.793	0.787	0.805	0.796	0.868	0.341	3677.811	0.060
4_가정반영	13	0.792	0.785	0.804	0.795	0.868	0.343	3673.968	0.000
0_기준선	7	0.749	0.741	0.766	0.753	0.824	0.258	4129.557	5.080
1_후진소거	6	0.750	0.742	0.767	0.754	0.824	0.258	4128.863	5.090
2_이상치대체	6	0.749	0.741	0.766	0.754	0.823	0.257	4136.292	5.170
3_정규화	6	0.749	0.741	0.766	0.754	0.823	0.257	4136.292	5.170

간명성 기준 1순위 : 5_가정반영+후진소거

성능 차이가 5% 이상이므로
고려대상 아님

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#10. 로지스틱 회귀 파이프라인

□ 4. 최종 모델 선택

◎ my_logit.compare_models()는 근소 격차 그룹 안에서 **변수가 하나 적은 `5_가정반영+후진 소거` 를 1순위로 지목**한다.

◎ 하지만 **5번이 지운 변수는 제곱항의 짝인 Crunchiness_c이므로 4번을 채택**한다.

◎ AIC 차이는 **1.93**(3673.97 → 3672.04)에 불과해, 변수 하나를 지워 얻는 이득이 거의 없다.

◎ **간명성 규칙이 통계적 원칙보다 앞설 수는 없다.**

○ 자동 선택 결과는 언제나 분석자가 검토해야 한다.

```python
# 계층 원칙에 따라 4번을 최종 채택한다
final_name = "4_가정반영"

final_fit = fits[final_name]
final_data = final_fit.data_        # 전처리가 끝난 데이터 (보고에 사용)

print("최종 모델 :", final_name)
print("독립변수  :", int(final_fit.df_model), "개")
```

✓ 마지막 실행: 오전 7:32:14, 기간: 0.0s Python

```
최종 모델 : 4_가정반영
독립변수  : 13 개
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



연습문제 사과 품질 분류 데이터셋 활용 로지스틱 회귀 #05

□ 분석 결과를 확인하고 다음 물음에 답하시오.

36. 성능 비교를 위해 정의한 전처리 체크포인트는 모두 몇 개인가요? (정수 1개)
37. 8개 체크포인트 전부에 공통으로 적용된 전처리 옵션은 무엇인가요? (옵션명 1개)
38. 가정 검정을 반영한 계열에서 후진소거법이 제거한 변수는 무엇인가요? (변수명 1개)
39. 정규화를 적용한 3번과 적용하지 않은 2번을 비교하면, 성능 지표 중 값이 달라지는 것은 몇 개인가요? (정수 1개)
40. AIC가 가장 낮은(가장 좋은) 모델의 이름은 무엇인가요? (모델명 1개)
41. compare_models()가 1순위로 지목한 모델을 그대로 쓰지 않고, 실제로 최종 채택한 모델은 무엇인가요? (모델명 1개)
42. 두 모델의 AIC 차이는 얼마인가요? (소수 둘째 자리)

6. 결과 보고

모든 실습이 하나의 노트북 파일에서 이어서 진행됩니다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#11. 최종 채택 모델에 대한 결과 보고

실습

1. 모형 적합도

```
1 Markdown(my_logit.report_fitness(final_fit))
```

✓ 마지막 실행: 오전 7:32:14, 기간: 0.0s Python

Note. n = 4000. LL = -1822.984, LL-Null = -2772.581, LLR $\chi^2(13)$ = 1899.193, p < 0.001, Pseudo R^2 = 0.342

Quality를 종속변수로, Sweetness, Weight_c, Weight_c2, Crunchiness_c, Crunchiness_c2, Juiciness_c, Juiciness_c2, Acidity_c, Acidity_c2, Ripeness_c, Ripeness_c2, Size_c, Size_c2(을)를 독립변수로 한 로지스틱 회귀분석 결과, 모형은 통계적으로 유의하였다.

> LLR $\chi^2(13)$ = 1899.193, p < 0.001, Pseudo R^2 = 0.342.

즉, 모형의 유사결정계수는 0.342로 매우 우수한 적합 수준을 보였다.

> ※ Pseudo R^2는 선형회귀의 R^2처럼 '분산 설명 비율'로 해석하지 않는다. 일반적으로 **0.2~0.4** 구간이면 매우 우수한 적합으로 본다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#11. 최종 채택 모델에 대한 결과 보고

실습

2. 독립변수 보고

```
1 display(my_logit.report_variables(final_fit, final_data))
2 Markdown(my_logit.report_variables_text(final_fit, data=final_data))
```

✓ 마지막 실행: 오전 7:32:14, 기간: 0.0s Python

	종속변수	독립변수	B	βstd	표준오차	z	유의확률	오즈비(OR)	OR 95% 하한	OR 95% 상한	공차	VIF
0	Quality	Size_c	0.629	1.214	0.031	20.410	0.000	1.876	1.766	1.993	0.664	1.506
1	Quality	Sweetness	0.568	1.105	0.030	18.806	0.000	1.766	1.664	1.873	0.646	1.547
2	Quality	Juiciness_c	0.485	0.937	0.027	17.805	0.000	1.625	1.540	1.714	0.787	1.270
3	Quality	Acidity_c	-0.274	-0.578	0.023	-11.889	0.000	0.760	0.727	0.795	0.831	1.203
4	Quality	Crunchiness_c2	0.192	0.625	0.019	10.399	0.000	1.212	1.169	1.257	0.922	1.084
5	Quality	Ripeness_c	-0.184	-0.345	0.029	-6.378	0.000	0.832	0.786	0.880	0.622	1.609
6	Quality	Weight_c2	0.176	0.693	0.015	11.948	0.000	1.192	1.158	1.227	0.951	1.051
7	Quality	Weight_c	0.165	0.264	0.033	4.945	0.000	1.179	1.105	1.259	0.702	1.424
8	Quality	Juiciness_c2	-0.103	-0.547	0.010	-9.991	0.000	0.902	0.884	0.920	0.884	1.131
9	Quality	Acidity_c2	-0.061	-0.375	0.008	-7.590	0.000	0.941	0.926	0.956	0.893	1.120
10	Quality	Ripeness_c2	-0.031	-0.150	0.010	-3.025	0.002	0.970	0.950	0.989	0.933	1.072
11	Quality	Size_c2	0.029	0.149	0.010	3.021	0.003	1.029	1.010	1.049	0.904	1.106
12	Quality	Crunchiness_c	0.010	0.014	0.036	0.270	0.787	1.010	0.941	1.084	0.797	1.254

독립변수 설명 텍스트 출력은 생략합니다.

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측

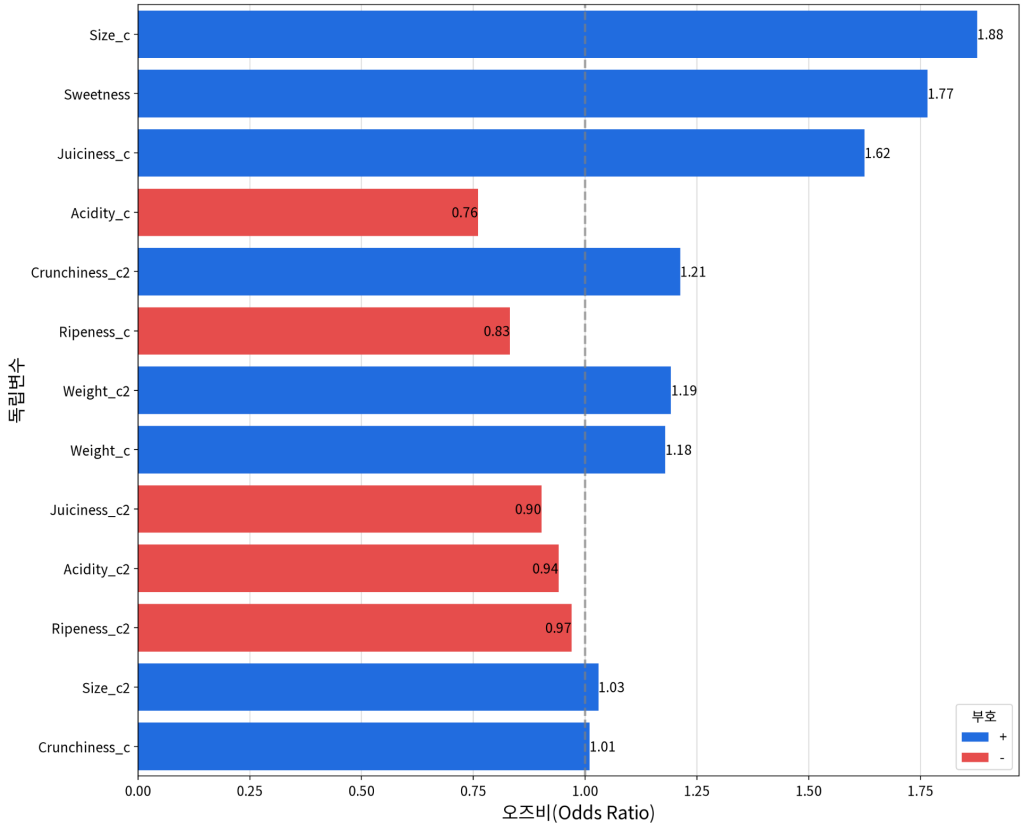
#11. 최종 채택 모델에 대한 결과 보고

실습

3. 오즈비 시각화

```
1 my_logit.plot_odds(final_fit, final_data)
```

✓ 마지막 실행: 오전 7:32:15, 기간: 0.1s Python



1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#11. 최종 채택 모델에 대한 결과 보고

실습

□ 인사이트 (1/2) — 계수 해석

모형 요약 — `logit(P(Quality = good)) ~ Sweetness + (Size·Weight·Crunchiness·Juiciness·Ripeness·Acidity의 1차항 + 제곱항)`

- **LLR $\chi^2(13) = 1{,}899.19$, $p < .001$, Pseudo $R^2 = 0.342$** 로 매우 우수한 적합이다. 13개 항 중 12개가 유의하다.
- **영향력 1위는 Size**다. 표준화 계수 βstd = 1.214로 2위 Sweetness(1.105)와 근소하고, |이어서 Juiciness(0.937) 순이다. → **크기·단맛·과즙이 품질을 결정하는 3대 축**이다.

1차항만 있는 변수 (직선 관계)

변수	오즈비	해석
Sweetness	**1.766**	단맛 1점 ↑ → good일 오즈 76.6% 증가

주의 — 제곱항이 있는 변수는 오즈비를 한 줄로 해석하지 않는다.
"1차항과 제곱항 두 계수가 함께 곡선을 만든다"라고 명시해야 함

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#11. 최종 채택 모델에 대한 결과 보고

□ 인사이트 (2/2) — 계수 해석 : 곡선 관계

1차항 + 제곱항이 있는 변수 (곡선 관계) — 두 계수가 함께 곡선을 만든다

변수	1차항 B	제곱항 B	곡선 모양	해석
Size	+0.629	+0.029	아래로 볼록 ∪	클수록 좋고, 커질수록 가속된다
Weight	+0.165	+0.176	아래로 볼록 ∪	평균에서 양쪽으로 멀어질수록 좋다
Crunchiness	+0.010 (n.s.)	+0.192	아래로 볼록 ∪	아주 아삭하거나 아주 무른 극단이 좋다
Juiciness	+0.485	−0.103	위로 볼록 ∩	많을수록 좋지만 어느 지점부터 꺾인다
Ripeness	−0.184	−0.031	위로 볼록 ∩	덜 익을수록 좋고, 과숙일수록 급격히 나빠진다
Acidity	−0.274	−0.061	위로 볼록 ∩	산도가 높을수록 나쁘고, 높아질수록 가속된다

- **Crunchiness가 이 분석의 가장 큰 반전이다.** T검정 p = 0.985(완전 무관), 기준선 모형 p = 0.253(제거 대상)이었던 변수가 **제곱항을 넣자 z = 10.40으로 세 번째로 강한 예측 변수**가 됐다.
- 1차항으로만 보면 효과가 없는 이유는, **양 극단이 모두 good 쪽이라 평균적으로 상쇄**되기 때문이다.
- EDA와 모델링 단계에서 각각 다른 이유로 놓칠 수 있었던 변수다.
  - EDA(T검정) → 평균 비교라서 U자 관계를 못 본다.
  - 기준선 모형 → 직선만 표현하므로 역시 못 본다.
  - **가정 검정(Box-Tidwell)에서만 잡힌다.**

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측

#11. 최종 채택 모델에 대한 결과 보고

4. 분류 성능 평가

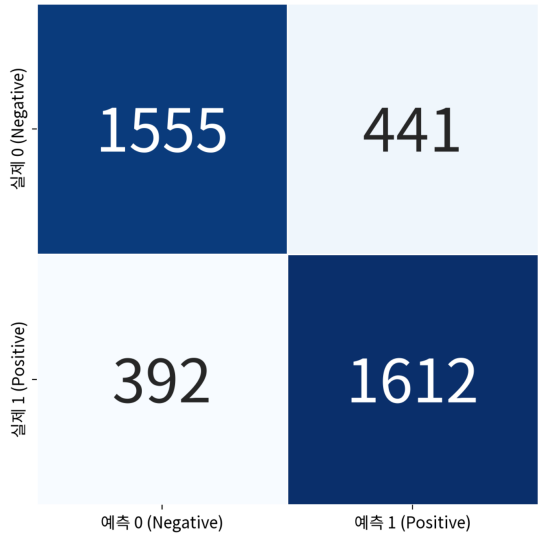
```
1 my_logit.report_performance(final_fit)
```

✓ 마지막 실행: 오전 7:32:18, 기간: 3.5s Python

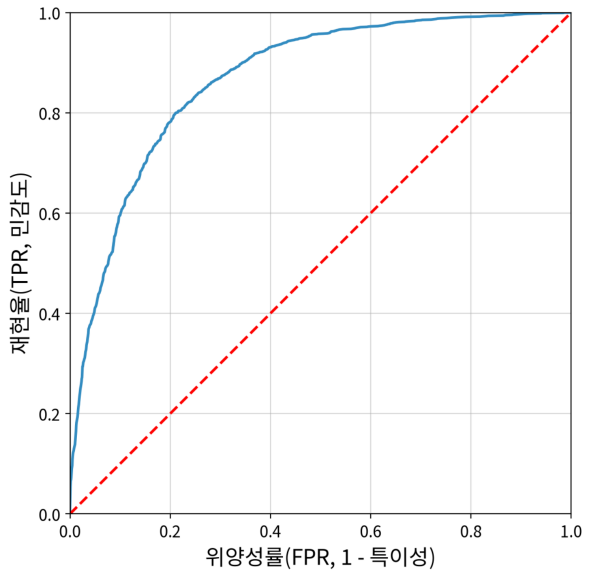
	예측 0 (Negative)	예측 1 (Positive)
실제 0 (Negative)	1555	441
실제 1 (Positive)	392	1612

	정확도(Accuracy)	정밀도(Precision)	재현율(Recall,TPR)	위양성율(Fallout,FPR)	특이성(Specificity,TNR)	F1	AUC	AUC 판단	진단오즈비(DOR)
performance	0.792	0.785	0.804	0.221	0.779	0.795	0.868	우수	14.500

혼동행렬(Confusion Matrix)



ROC Curve (AUC = 0.8682)



인사이트

- **정확도는 0.5에서 최대(0.792)**지만, **F1은 0.4에서 최대(0.802)**다.
- **트레이드오프가 그대로 나타난다.** 0.3 → 재현율 0.932 / 정밀도 0.699, 0.7 → 정밀도 0.857 / 재현율 0.594.
- **정답은 목적이 정한다.**
  - 프리미엄 선별 → 임계값 ↑ (정밀도)
  - 폐기 걸러내기 → 임계값 ↓ (재현율)

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-32)

실습

□ 임계값 비교 함수 정의

```
699 def report_threshold(fit, thresholds=None, digits=3):
700     """임계값을 바꿔가며 분류 성능이 어떻게 달라지는지 한 표로 정리한다.
701
702     임계값 0.5는 관례일 뿐이므로, 정밀도와 재현율 중 무엇이 중요한지에 따라
703     조정한 결과를 비교해 목적에 맞는 값을 고르는 데 사용한다.
704
705     Args:
706         fit: `fit_model` 함수로 적합된 회귀분석 결과 객체.
707         thresholds (list): 비교할 임계값 목록 (기본값: [0.3, 0.4, 0.5, 0.6, 0.7]).
708         digits (int): 표에 표시할 소수점 자릿수 (기본값: 3).
709         report (bool): 비교표를 화면에 출력할지 여부 (기본값: True).
710
711     Returns:
712         DataFrame: 임계값을 인덱스로 하는 성능 비교표.
713             정확도·정밀도·재현율·F1 과 놓친 1(FN)·잘못 잡은 0(FP) 건수를 담는다.
714
715     Raises:
716         ValueError: 임계값 목록이 비었거나 0~1 범위를 벗어난 값이 있는 경우.
717     """
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit (step-33)

실습

□ 구현 단계 파이프라인

```
699 def report_threshold(fit, thresholds=None, digits=3):
700     """임계값을 바꿔가며 분류 성능이 어떻게 달라지는지 한 표로 정리한다.…
718     # --- 1) 입력 검증 ---
719
720     # --- 2) 실제값·예측확률 준비 ---
721
722     # --- 3) 임계값별 성능 계산 ---
723
724
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT]
사과 품질 분류
7. [PBT]
타이타닉 생존률 예측



/helpers/my_logit (step-34)

실습

□ 입력값 검증 / 실제 값 및 확률 준비

```
718     # --- 1) 입력 검증 ---
719     if thresholds is None:
720         thresholds = [0.3, 0.4, 0.5, 0.6, 0.7]
721
722     if not len(thresholds):
723         raise ValueError("비교할 임계값이 없습니다.")
724
725     for t in thresholds:
726         if not 0 < t < 1:
727             raise ValueError(f"임계값은 0과 1 사이여야 합니다: {t}")
728
729     # --- 2) 실제값·예측확률 준비 ---
730     y_true = np.asarray(fit.model.endog).astype(int)    # 실제 종속변수(0/1)
731     proba = np.asarray(fit.predict())                   # 1이 될 확률
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



/helpers/my_logit (step-35)

실습

□ 임계값에 따른 성능 계산

```
733     # --- 3) 임계값별 성능 계산 ---
734     rows = []
735
736     for t in thresholds:
737         y_pred = (proba > t).astype(int)                # 임계값 t 로 이진화
738         tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
739
740         rows.append({
741             "임계값": t,
742             "정확도": accuracy_score(y_true, y_pred),
743             "정밀도": precision_score(y_true, y_pred, zero_division=0),
744             "재현율": recall_score(y_true, y_pred, zero_division=0),
745             "F1": f1_score(y_true, y_pred, zero_division=0),
746             "놓친 1(FN)": int(fn),       # 임계값을 올릴수록 늘어난다
747             "잘못 잡은 0(FP)": int(fp),  # 임계값을 내릴수록 늘어난다
748         })
749
750     result = DataFrame(rows).set_index("임계값").round(digits)
751
752     return result
```

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



#11. 최종 채택 모델에 대한 결과 보고

실습

5. 임계값 조정

◎ 임계값 0.5는 관례일 뿐 정답이 아니다. 목적에 따라 조정한다.

```
1 my_logit.report_threshold(final_fit)
```

✓ 마지막 실행: 오전 8:25:51, 기간: 0.0s Python

임계값	정확도	정밀도	재현율	F1	놓친 1(FN)	잘못 잡은 0(FP)
0.300	0.765	0.699	0.932	0.799	137	803
0.400	0.784	0.740	0.875	0.802	250	615
0.500	0.792	0.785	0.804	0.795	392	441
0.600	0.779	0.822	0.713	0.764	575	309
0.700	0.747	0.857	0.594	0.702	813	198

인사이트

- **정확도는 0.5에서 최대(0.792)**지만, **F1은 0.4에서 최대(0.802)**다.
- **트레이드오프가 그대로 나타난다.** 임계값 0.3 → 재현율 0.932 / 정밀도 0.699, 임계값 0.7 → 정밀도 0.857 / 재현율 0.594.
- **정답은 목적이 정한다.**
  - **프리미엄 등급 선별**(내보내는 사과는 반드시 good이어야 함) → 임계값을 **높여** 정밀도 확보
  - **폐기 대상 걸러내기**(좋은 사과를 버리면 손해) → 임계값을 **낮춰** 재현율 확보

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



한계점

- 처방을 고르는 절차가 자동 규칙에 의존한다. **제곱항과 로그 변환 중 AIC가 낮은 쪽을 고르게 했지만, 원칙은** 로짓 대비 산점도의 모양을 보고 판단**하는 것이다.**
  이 데이터는 7종 모두 음수를 포함해 로그 변환이 후보에서 빠졌으므로 사실상 제곱항 외의 선택지가 없었다.
- 제곱항을 6개 추가하면서 모형이 복잡해졌다.
  곡선을 표현할 수 있게 된 대신 변수당 계수가 2개로 늘어 오즈비를 한 줄로 해석할 수 없게 됐고, **과적합 위험도 함께 올라갔다.**
- 성능을 학습 데이터에서 그대로 측정했다.
  훈련·검증 데이터를 나누지 않았으므로 정확도 0.792와 AUC 0.868은 낙관적인 값**이다.**
- 변수의 물리적 의미를 알 수 없다. **값이 이미 표준화된 상태로 제공되어 "크기 1점"이 몇 mm인지, "산도 1점"이 pH 얼마인지 알 수 없다.** 오즈비를 현장 단위로 환산할 수 없다.
- 품질 라벨의 기준이 공개되어 있지 않다. **good/bad를 누가 어떤 기준으로 판정했는지 모르므로, 모형이 학습한 것은 "사과의 객관적 품질"이 아니라** "이 데이터를 만든 사람의 판정 기준"**일 수 있다.**
- 인과관계가 아니라 연관관계다. **산도를 인위적으로 낮춘다고 사과가 good이 되지는 않는다.**

ITWILL 이광호강사

1. 로지스틱 회귀의 이해
2. 로지스틱 회귀의 해석
3. 다중 로지스틱 회귀
4. 분류 성능 평가
5. 로지스틱 회귀의 가정
6. [PBT] 사과 품질 분류
7. [PBT] 타이타닉 생존률 예측



연습문제 사과 품질 분류 데이터셋 활용 로지스틱 회귀 #06

□ 결과 보고 내용을 확인하고 다음 물음에 답하시오.

43. 최종 채택 모델의 독립변수는 몇 개인가요? (정수 1개)
44. 모형 적합도 보고에 제시된 Pseudo R^2는 얼마인가요? (소수 셋째 자리)
45. 13개 독립변수 중 유의수준 5% 기준으로 유의하지 않은 것은 무엇인가요? (변수명 1개)
46. 최종 모형에서 표준화 계수(βstd)가 가장 큰 변수는 무엇인가요? (변수명 1개)
47. T검정에서 p = 0.985로 완전 무관, 기준선 모형에서도 p = 0.253으로 제거 후보였다가, 제곱항을 넣자 z = 10.40으로 강력한 예측 변수가 된 변수는 무엇인가요? (변수명 1개)
48. 최종 모형의 분류 정확도는 얼마인가요? (소수 셋째 자리)
49. 진단 오즈비(DOR)는 얼마인가요? (소수 첫째 자리)
50. 혼동행렬에서, 실제로는 good인데 모형이 bad로 잘못 판정해 놓친 사과는 몇 개인가요? (정수 1개)
51. 임계값을 0.3 ~ 0.7로 바꿔가며 비교했을 때, F1이 가장 높아지는 임계값은 얼마인가요? (소수 첫째 자리)
52. 내보내는 사과가 반드시 good이어야 하는 프리미엄 등급 선별에서는 임계값을 높입니다. 이때 확보하려는 지표는 무엇인가요? (지표 이름 1개)
53. 훈련·검증 데이터를 나누지 않고 학습 데이터에서 그대로 성능을 측정했습니다. 제곱항 6개를 추가하면서 함께 커진 위험은 무엇인가요? (용어 1개)
54. "산도가 낮을수록 good이다"라는 결과를 두고 "산도를 인위적으로 낮추면 사과가 좋아진다"고 말할 수는 없습니다. 이 분석이 밝힌 것은 인과관계가 아니라 무엇인가요? (용어 1개)